

LibAFL-DiFuzz: Advanced Architecture Enabling Directed Fuzzing

Darya Parygina,
Timofey Mezhuev,
Daniil Kuts

December 12, 2024

ISP RAS, Sydr-Fuzz team

Directed fuzzing is a kind of fuzzing that focuses on reaching predefined program location with the help of some metrics.

- **Coverage-guided fuzzing** becomes a prevailing dynamic testing approach in SSDLC
- **Directed fuzzing** helps to achieve better results in patch testing, crash reproduction, static analysis report verification
- Directed fuzzing assesses the helpfulness of the inputs in reaching predefined program points using **proximity metrics**
- Directed approach gains higher **effectiveness** and **speed** of errors detection, provides **flexibility** in proximity metrics definition

Existing Approaches to Directed Fuzzing

AFLGo – the pioneer of directed fuzzing:

- **Weighted distance** to target points as proximity metric
- Target points reachability as optimization problem
- Power scheduling based on **simulated annealing** algorithm

Enhancements: dynamic instrumentation, **target sequences-based metrics**, **pruning irrelevant code**, context-aware analysis, adaptive mutations, **DSE integration**.

Common problems:

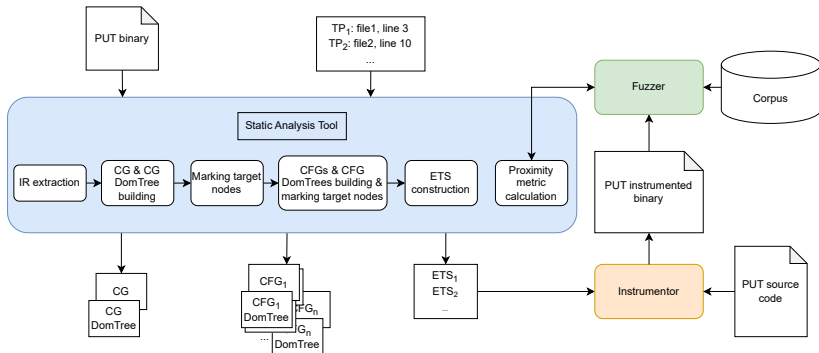
- **Insufficient** program structure analysis
- **Ignoring context** of target points, **indirect transfers**, data flow
- **Inaccurate** proximity metrics calculation
- Lack of flexibility and configurability

Target sequences-based power scheduling for directed fuzzing within LibAFL modular architecture.

Method features:

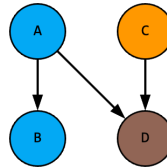
- Indirect edges resolving for call graph refining in directed fuzzing
- Proximity metrics based on target sequences coverage enhanced with context weights
- Power scheduling based on combination of metrics:
 1. proximity metrics
 2. overall coverage growth metric
- Simulated annealing algorithm
- Flexible, configurable, modular architecture for directed fuzzing

Directed Fuzzing Pipeline

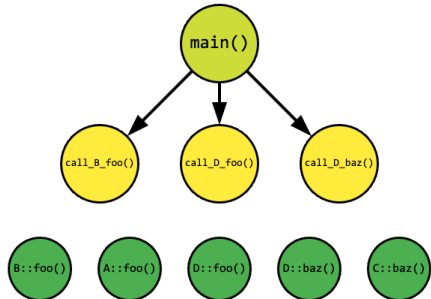


Static stage: CG and CFG construction

```
1  struct A {  
2      virtual void foo();  
3  };  
4  struct B : A {  
5      virtual void foo();  
6  };  
7  struct C {  
8      virtual void baz();  
9  };  
10 struct D: C, A {  
11     virtual void baz();  
12     virtual void foo();  
13 };
```

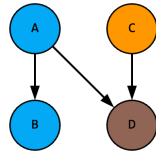


Initial call graph with unlinked components:

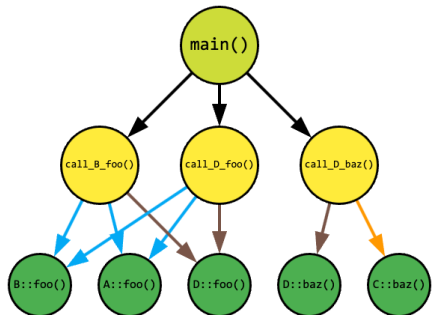


Static stage: CG and CFG construction

```
1  void call_B_foo(){
2      A *b = new B();
3      b->foo();
4      delete b;
5  }
6
7  void call_D_baz(){
8      C *dc = new D();
9      dc->baz();
10     delete dc;
11 }
12
13 void call_D_foo(){
14     A *da = new D();
15     da->foo();
16     delete da;
17 }
```



Call graph with indirect edges refined based on pointer type:



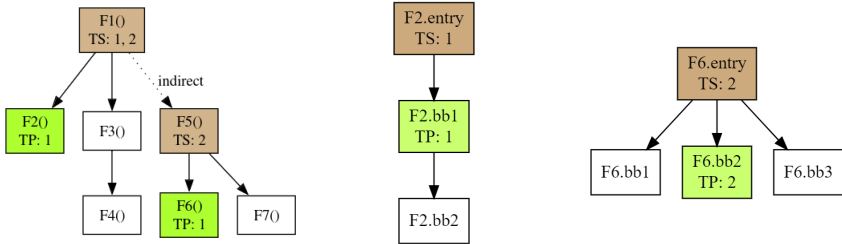
Static stage: ETS

ETS is an enhanced target sequence that contains:

- **all function nodes** that are necessarily called before the target function
- all **BB nodes inside the target function**, which are necessarily executed before the target point

$$ETS_{F_2} = \{F1, F2, F2.entry, F2.bb1\}$$

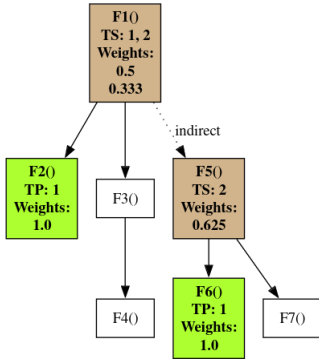
$$ETS_{F_6} = \{F1, F5, F6, F6.entry, F6.bb2\}$$



Static stage: context weights

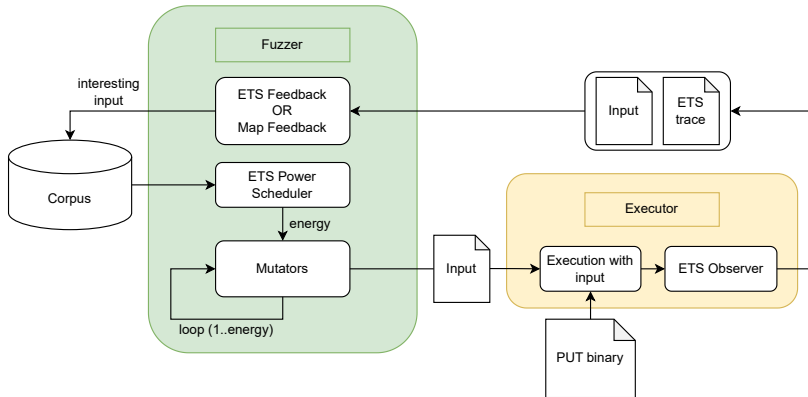
Context weight for node i relative to ETS_k :

$$weight_{ETS_k}(i) = \frac{(Distance(i, t) + Level(i) + Successors(i) + P_i)}{4}$$



- $Distance(i, t)$: shortest distance from node i to target point t
- $Level(i)$: level in domtree
- $Successors(i)$: relevant successors in graph
- $Probability(i)$: probability of path in domtree

Dynamic stage: fuzzing instances



Dynamic stage: proximity metrics

- $PriorityW_i$: number of ETSs similar to the given ETS_i
- $SeqCovW_i(T, ETS_i)$: similarity between execution trace T and given ETS_i
- $gMaxCovW_i$: largest historical value of $SeqCovW_i$ for ETS_i

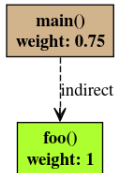
Comprehensive factor (N is the number of ETSs):

$$CFW = \begin{cases} \frac{1}{2} (SeqCovW + PriorityW / N), & \text{if } \sum_{j=1}^N (gMaxCovW_j \geq \beta ? 1 : 0) < \frac{N}{2} \\ \frac{1}{3} (SeqCovW + PriorityW / N + (1 - gMaxCovW)), & \text{if } \sum_{j=1}^N (gMaxCovW_j \geq \beta ? 1 : 0) \geq \frac{N}{2} \end{cases}$$

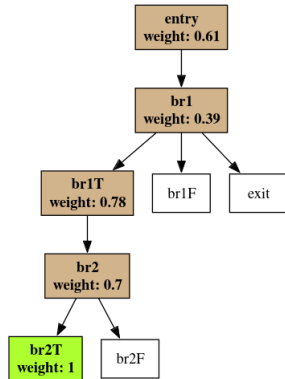
Combines all metrics into one value representing execution trace proximity to reaching target point

Dynamic stage: proximity metrics

Call graph domtree



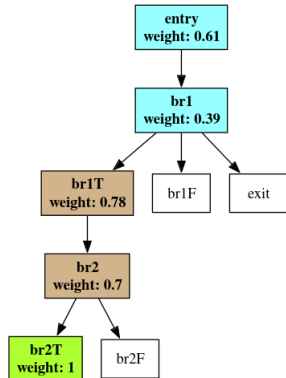
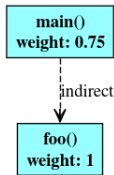
CFG domtree for function foo()



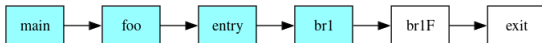
Dynamic stage: proximity metrics

CFG domtree for function foo()

Call graph domtree



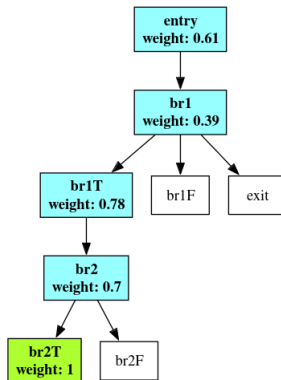
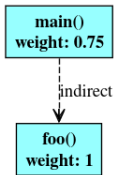
Trace T_1



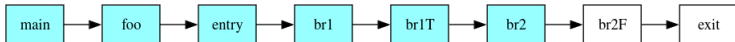
Dynamic stage: proximity metrics

CFG domtree for function foo()

Call graph domtree



Trace T_2



Implementation

- **DiFuzzLLVM**: library in C++
 - graph construction based on LLVM IR
 - indirect calls resolving
- **DiFuzz**: tool in Rust
 - dominator trees construction
 - mapping between target points and graph nodes based on config
 - building ETS for each target point
 - context weights calculation
 - proximity metrics calculation during fuzzing
- **Static instrumentation**: ETS-related LLVM pass
- **Fuzzer**: LibAFL-based tool
 - ETS Observer: saving ETS trace
 - ETS Feedback: detecting inputs with unique ETS traces
 - ETS Reached Feedback: detecting reaching target points
 - ETS PowerScheduler: power scheduling based on proximity metrics

Experiments: TTE

Crashes:

ID	Target	Crash type	Target point
1	giflib	Null pointer dereference	util/gifsp sponge.c: 61
2	jasper	Abort	src/libjasper/ mif/mif_cod.c:491
3	libxml	Null pointer dereference	valid.c:1181
4	libxml	Null pointer dereference	valid.c:1189

Time-to-Exposure results vs AFLGo:

ID	Target	LibAFL-DiFuzz	AFLGo	Speedup
1	giflib	3.9 s	7.5 s	1.92
2	jasper	66 s	240 s	3.64
3	libxml	16 s	180 s	11.25
4	libxml	1554 s	390 s	0.25

Experiments: TTE

Time-to-Exposure results vs BEACON:

ID	Target	LibAFL-DiFuzz	BEACON	Speedup
1	giflib	3.9 s	5.5 s	1.41
2	jasper	66 s	150 s	2.72
3	libxml	16 s	150 s	9.38
4	libxml	1554 s	140 s	0.09

Time-to-Exposure results vs SieveFuzz:

ID	Target	LibAFL-DiFuzz	SieveFuzz	Speedup
1	giflib	3.9 s	13.0 s	3.33
2	jasper	66 s	30 m TO	–
3	libxml	16 s	30 m TO	–
4	libxml	1554 s	30 m TO	–

Experiments: total crashes number

ID	Target	LibAFL-DiFuzz	AFLGo	BEACON
1	giflib	605	524	706
2	jasper	9340	$\approx 397k$	$\approx 39k$
3	libxml	46	2003	203

Why LibAFL-DiFuzz?

- Configurability, reaching targets reporting, directed fuzzing statistics:
 - flexible LibAFL architecture allows to interchange modules from different approaches, combine them together
 - ETSReachedFeedback allows to detect reaching target point by meeting location in code (not only crash)
- Multi-targeted directed fuzzing:
 - presented in AFLGo
 - unsupported in state-of-the-art BEACON, SieveFuzz
 - supported in LibAFL-DiFuzz

Conclusion

Implemented target sequences-based power scheduling for directed fuzzing within LibAFL modular architecture.

- Improved call graph construction with indirect edges resolving
- Enhanced target sequences with context weights for each sequence component
- Implemented proximity metrics calculation based on target sequences coverage
- Implemented power scheduling based on simulated annealing algorithm
- Built flexible, easy to configure architecture for directed fuzzing within LibAFL library

Further plans

- Prioritization of inputs based on proximity metrics
- Coordination of exploration-exploitation modes
- Pruning irrelevant code parts
- Integration with Sydr dynamic symbolic execution tool via input queues

Thank you! Questions?